
Hyperparameter Optimization in the Age of Large Models

Ali Elganzory*
University of Freiburg
elganzo@cs.uni-freiburg.de

Erlis Lushtaku*
University of Freiburg
lushtake@cs.uni-freiburg.de

Abstract

Training large language models (LLMs) is both computationally expensive and sensitive to hyperparameter settings. Despite the acknowledged importance of hyperparameter optimization (HPO) in deep learning, its applicability to LLM pretraining remains underexplored—largely due to the prohibitive cost of experimentation. To address this, we study modern multi-fidelity HPO algorithms like Priorband, which are designed for constrained compute budgets, and compare them against strong baselines, including Maximal Update Parametrization (μ P) transfer and scaling up the model with a well-tuned default. Our results show that while advanced HPO methods offer performance gains, μ P transfer provides a highly effective and cheap baseline. Most critically, at the scales that we tested, our findings suggest that for a fixed compute budget, allocating resources to training a larger model yields greater performance improvements than investing that budget in HPO for a smaller model.

1 Introduction

The development of Large Language Models (LLMs) is currently guided by a well-established scaling paradigm: model performance, typically measured by test loss, improves as a power-law function of model size, dataset size, and the total computational budget [Kaplan et al., 2020, Brown et al., 2020]. This principle has driven the creation of models with hundreds of billions of parameters, achieving new levels of capability [Hoffmann et al., 2022]. However, this success has introduced a significant challenge. Hyperparameter optimization (HPO), the process of tuning parameters like learning rate, batch size, and regularization terms, is critical for realizing the full potential of any machine learning model but has become computationally infeasible for foundation models due to the immense cost of each training run [Hutter et al., 2019, Yu and Zhu, 2020].

To address this challenge, one promising alternative to exhaustive HPO has emerged: Maximal Update Parametrization (μ P). Proposed by Yang et al. [2021], μ P offers a principled method for hyperparameter transfer. By carefully re-parametrizing the model, μ P ensures that optimal hyperparameters—such as the learning rate—remain stable as the model scales in size. This allows practitioners to tune these settings on a small, inexpensive proxy model and then transfer them directly to a large target model, a technique known as μ P Transfer (or μ Transfer), drastically reducing the cost of finding a good configuration.

This tension between costly, empirical HPO and inexpensive, principled transfer with μ P creates a crucial dilemma for practitioners. Forced to choose, practitioners often rely on default hyperparameters from prior work or heuristics, potentially leaving performance gains on the table. In this work we evaluate whether HPO justifies its costs.

This investigation is guided by three primary research questions:

*Equal contribution.

- **Are existing HPO algorithms, particularly multi-fidelity methods, applicable and effective in the LLM pre-training setting?**

We conduct a systematic empirical comparison of modern HPO algorithms, showing that methods like Priorband outperform standard HPO baselines in the LLM pre-training regime.

- **How much performance benefit can HPO provide compared to the near-zero-shot μ P hyperparameter transfer method?**

We quantitatively compare HPO against μ P transfer, finding that the latter is a surprisingly strong and cost-effective baseline, though HPO can yield modest additional gains.

- **Given an unconstrained final model size, is it better to allocate additional compute to HPO or to training a larger model with default hyperparameters?**

We show that, for our tested scales, scaling the model size with a strong default inspired from open-sci-ref [Nezhurina et al., 2025a,b], yields larger performance improvements than HPO for a smaller model, reinforcing scaling-law predictions. However, we recommend investigating the behavior at higher scales, for we suspect that there may be an advantage for HPO the further we go down the loss-compute scaling curve [Hoffmann et al., 2022].

In addition, we make the following engineering contributions:

- **Compute-as-Fidelity Framework:** A novel methodology that enables the use of the raw compute budget (measured in FLOPs) as a fidelity parameter for multi-fidelity HPO algorithms, enabling the usage of total compute as a budget for HPO algorithms. This is achieved through a deterministic, scaling-law-based translation from a given budget (in FLOPs) to a specific, compute-optimal LLM architecture.
- **Reproducible Tooling:** A suite of automated and deterministic tooling built upon established frameworks, namely Nvidia Megatron-LM [Shoeybi et al., 2019] and NePS [Stoll et al., 2024], to ensure reproducibility and efficiency in large-scale, parallel HPO experiments.

2 Background and Related Works

Our work is situated at the intersection of three key research areas: neural scaling laws, modern hyperparameter optimization (HPO), and principled hyperparameter transfer.

The Evolution of Scaling Laws for Language Models. Foundational work on neural scaling laws established that the performance of language models scales predictably as a power-law function of compute, data, and model size [Kaplan et al., 2020, Brown et al., 2020]. This paradigm was refined by Hoffmann et al. [2022], who demonstrated that for compute-optimal training, model and dataset size should be scaled in tandem. More recently, these laws have been extended to also account for hyperparameters like batch size [Bi et al., 2024]. While these principles provide a valuable blueprint for how to scale, they often presuppose a set of well-chosen hyperparameters. Our work addresses the practical challenge of finding these hyperparameters, treating the scaling laws not as an endpoint, but as a tool to define a compute-centric framework for evaluating different optimization strategies.

Modern Hyperparameter Optimization. The practice of HPO has matured significantly from early, inefficient methods like Grid Search and Random Search [Bergstra and Bengio, 2012] to more sample-efficient strategies like Bayesian Optimization [Snoek et al., 2012]. Given the prohibitive cost of training large models, multi-fidelity optimization methods have become particularly relevant. These methods, such as Hyperband [Li et al., 2018], leverage low-cost approximations (i.e., lower fidelities) to rapidly discard unpromising hyperparameter configurations. Priorband [Mallik et al., 2023] further enhances this by incorporating prior knowledge into the search process. However, the applicability and efficacy of these advanced methods in the specific context of LLM pre-training remain underexplored. Our work bridges this gap by systematically evaluating modern multi-fidelity algorithms against other common strategies, using the total computational budget itself as a natural fidelity.

Hyperparameter Transfer with Maximal Update Parametrization (μ P). As a direct alternative to costly empirical HPO, Maximal Update Parametrization (μ P) provides a principled method for transferring hyperparameters across model scales [Yang et al., 2021]. By re-parametrizing the network, μ P ensures that the optimal hyperparameters of a small, inexpensive model remain stable for a much larger target model, enabling a near-zero-shot transfer. This presents a powerful and

theoretically grounded baseline. While the benefits of μ P are well-established, prior work has not quantitatively benchmarked its effectiveness against a comprehensive, multi-fidelity HPO search. This motivates a central question of our study: how large is the performance gap between this inexpensive transfer method and a more exhaustive, empirical search? To our knowledge, no prior study has conducted a direct comparison of modern HPO methods against both μ P transfer and the pragmatic strategy of simply scaling up a model with a robust default configuration. Our work aims to fill this void, providing practitioners with empirical guidance on the crucial decision of how to best allocate a fixed computational budget.

3 A Compute-Centric Framework for HPO

A fundamental challenge in applying multi-fidelity HPO to LLMs is defining a meaningful fidelity. Multi-fidelity algorithms presuppose the existence of a fidelity parameter that represents the cost of an evaluation. Since the computational budget (measured in FLOPs) is a natural and direct representation of the cost of pre-training, we use the budget itself as the fidelity. By leveraging scaling laws, we can deterministically map any given compute budget to a unique, compute-optimal model architecture.

3.1 From Compute Budget to Model Architecture

Our framework provides a deterministic translation layer that takes a compute budget C (in FLOPs) and outputs a complete set of architectural parameters. This process is governed by a series of equations derived from established scaling laws and architectural best practices [Kaplan et al., 2020, Hoffmann et al., 2022, Bi et al., 2024].

First, the total number of parameters, N_{params} , is estimated. This includes both non-embedding and embedding parameters, following an adaptation from Kaplan et al. [2020], which includes embeddings as advised by Hoffmann et al. [2022]. The calculation is based on the model’s hidden size (d_{model}), number of layers (n_{layer}), attention head dimension (d_{attn}), vocabulary size (n_{vocab}), and sequence length (l_{seq}). The feed-forward hidden size (d_{ff}) is set to $4 \times d_{model}$. The total parameter count is given by:

$$N_{params} = \underbrace{n_{layer} \cdot (d_{model} \cdot 3d_{attn} + d_{attn}d_{model} + 8d_{model}^2)}_{\text{Non-embedding}} + \underbrace{(n_{vocab} + l_{seq}) \cdot d_{model}}_{\text{Embedding}} \quad (1)$$

Second, to maintain compute-optimal training, the number of training tokens, T , is scaled linearly with the number of parameters, following the Chinchilla finding [Hoffmann et al., 2022]:

$$T = 20 \times N_{params} \quad (2)$$

Third, the computational cost per token, M , is estimated using the formula proposed by Bi et al. [2024], which accounts for both the feed-forward and attention computations:

$$M \approx 72 \cdot n_{layer} \cdot d_{model}^2 + 12 \cdot n_{layer} \cdot d_{model} \cdot l_{seq} \quad (3)$$

The total compute budget is the product of tokens and FLOPs-per-token, $C = T \times M$. By substituting Equations 1, 2, and 3 and incorporating architectural constraints (e.g., a well-performing range of the aspect ratio between d_{model} and n_{layer}) [Kaplan et al., 2020], we arrive at a single polynomial equation in n_{layer} . For any given compute budget C , this equation can be solved to find the corresponding n_{layer} , from which all other architectural parameters are derived.

3.2 Experimental Setup

All models were pre-trained on a 6 billion token subset of the SlimPajama dataset [?], which is a deduplicated and filtered version of the RedPajama corpus. The dataset was split into 96.9% for training, 3.0% for validation, and 0.1% for testing. HPO experiments were orchestrated using the Neural Pipeline Search (NePS) framework, chosen for its robust support for prior-informed and multi-fidelity optimization tailored to deep learning [Stoll et al., 2024, Mallik et al., 2023].

We evaluated the following algorithms and baselines:

- **Grid and Random Search:** Standard baseline HPO methods [Bergstra and Bengio, 2012].

- **Hyperband:** A multi-fidelity method [Li et al., 2018].
- **Priorband:** An extension of Hyperband that incorporates prior beliefs [Mallik et al., 2023].
- **μ P Transfer:** A strong scaling method where hyperparameters are tuned on a small proxy model and transferred to the larger target model using μ P [Yang et al., 2021].
- **Scaling Up:** The "scale over tune" strategy, where a default hyperparameter configuration is used to train a larger model.

The target model size for all methods, except for the "Scaling Up" experiment, is 141.8 million (M) parameters. The target for the "Scaling Up" experiment is 200.6M parameters.

For multi-fidelity methods, the fidelities correspond to compute budgets that map to model sizes of 40.3M, 76.2M, and 141.8M parameters. The hyperparameter search space, based on recommendations from prior work [Nezhurina et al., 2025a,b], is detailed in Appendix A. The architectural parameters for each fidelity level are detailed in Appendix B [Kaplan et al., 2020].

For μ P, the smaller proxy model consisted of 11.16M parameters, while the larger target model matched the HPO methods' target in size.

3.3 Training Infrastructure and Automation

Our experiments were conducted on a high-performance computing cluster consisting of NVIDIA L40S GPUs. We utilized a fork of the Nvidia Megatron-LM framework [Shoeybi et al., 2019], a powerful tool for training large transformer models. We implemented both Standard Parametrization (SP) and Maximal Update Parametrization (μ P) [Yang et al., 2021], including base widths, parameter multipliers, and learning rate multipliers for different weight groups.

Reproducibility and efficiency in large-scale distributed experiments are paramount. To this end, we developed several automation utilities:

- **Deterministic Scheduler:** A key contribution is a worker scheduler that enforces fidelity-by-fidelity evaluation for multi-fidelity HPO algorithms. In asynchronous parallel settings, faster workers can cause race conditions that influence which configurations are promoted, introducing stochasticity. Our scheduler ensures that all configurations at a given fidelity level (or "rung") are completed before any are promoted to the next, making the HPO process deterministic and the results reproducible.
- **Adaptive Batch Sizing:** We automated the process of finding the maximum per-device micro-batch size to prevent out-of-memory errors during training.
- **Resource Balancing:** We implemented logic to balance the number of GPUs and gradient accumulation steps to efficiently match the target global batch size dictated by the scaling laws for a given model architecture.

4 Empirical Results and Discussion

In the next sections 4.1, 4.2, and 4.3, we present our empirical evaluation, which provides answers to the three research questions listed in the introduction, respectively. The primary results are summarized in Table 1, which compares the final validation loss across all methods and baselines, and Figure 1, which shows the incumbent performance for all of the experimented approaches.

4.1 The Efficacy of HPO Algorithms for LLMs

The results indicate that modern HPO algorithms are applicable and effective for LLM pre-training. As shown in Figure 1, the multi-fidelity methods, Hyperband and Priorband, converge to better solutions than Grid Search and Random Search. At a 6x budget, Priorband achieves a mean loss of 2.71 ± 0.00 , which is lower than Random Search's 2.77 ± 0.03 and Grid Search's 2.82 ± 0.01 (Table 1). This suggests that the intelligent resource allocation of multi-fidelity methods, which leverages cheaper, smaller model evaluations to prune the search space, is an effective strategy in the LLM context.

Table 1: Performance comparison across methods and budgets, each evaluated on two seeds (42 and 73). Performance is measured by the mean and standard error of the validation loss. Budget is a multiple of the compute for one max fidelity (141.8M parameter model) evaluation. We report results at 6x budget, which corresponds to the point where all HPO methods converged, although we ran them for at least 7x budget.

Method	Budget	Target Model Size	Mean $\pm$ Stderr
μ P Transferred Model	1x	141.8M	2.74 ± 0.01
Scaled Up Model	3x	200.6M	2.59 ± 0.01
Grid Search	3x	141.8M	2.82 ± 0.01
Random Search	3x	141.8M	2.83 ± 0.04
Grid Search	6x	141.8M	2.82 ± 0.01
Random Search	6x	141.8M	2.77 ± 0.03
Hyperband	6x	141.8M	2.75 ± 0.01
Priorband	6x	141.8M	2.71 ± 0.00
Priorband	20x	141.8M	2.71 ± 0.00

Furthermore, Priorband consistently outperforms Hyperband (2.71 ± 0.00 vs. 2.75 ± 0.01). This gap highlights the potential benefit of incorporating expert priors into the search process. By biasing its search towards configurations believed to be promising, Priorband finds better solutions faster.

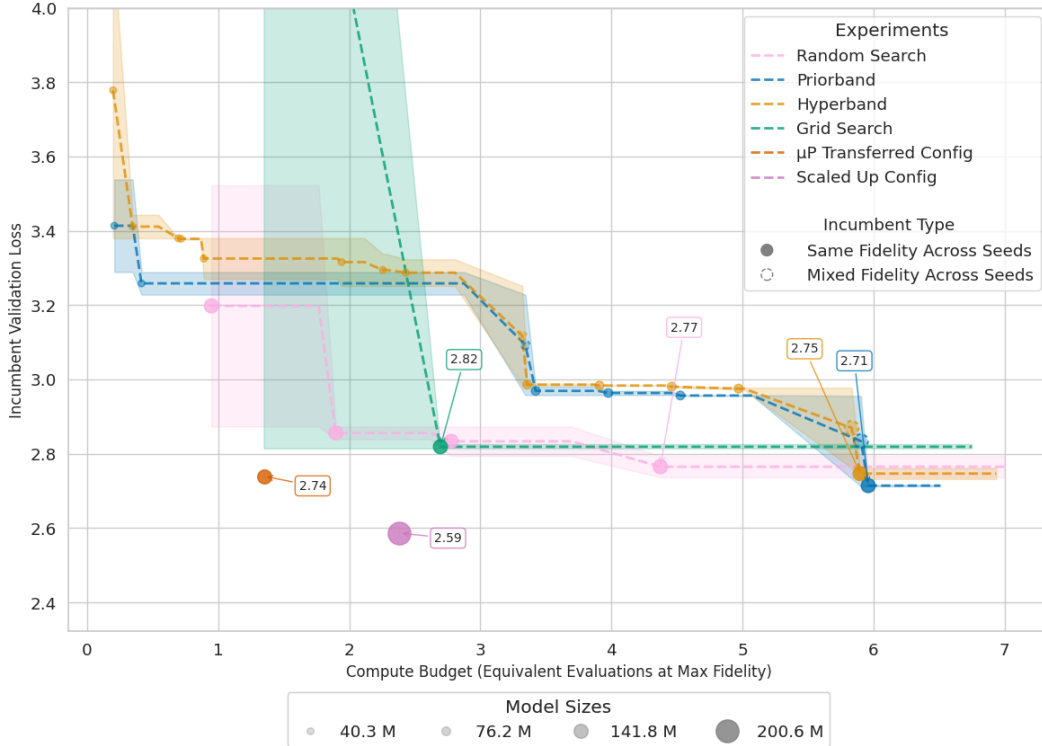


Figure 1: Mean incumbent validation loss and standard error versus computational budget for the different experiments. The budget is normalized by the cost of a single evaluation at the maximum fidelity (141.8M parameters) of multi-fidelity methods. Since the runtime is not exactly the same across the different evaluations, some incumbent values are calculated across different fidelities and are referred to as "Mixed Fidelity". While multi-fidelity methods (Priorband, Hyperband) find better solutions than naive search, a scaled-up model with default hyperparameters ultimately achieves the lowest validation loss.

4.2 Quantifying the Benefit of HPO over μ P Transfer

In this section, we compare the performance of a low-cost approach (μ P transfer) against a more computationally expensive one (HPO). The μ P transfer baseline achieves a loss of 2.74 ± 0.01 with a budget of only 1x (one full-fidelity training run plus the negligible cost of tuning the 11M-parameter proxy model). Notably, this result is competitive with Hyperband, which achieves a similar loss of 2.75 ± 0.01 but requires a significantly larger 6x budget.

However, the best-performing HPO method, Priorband, reached a final loss of 2.71 ± 0.00 (at a 6x budget). This indicates that while μ P transfer provides a very strong and cost-effective starting point, further performance gains may be unlocked through HPO. The decision to pursue these gains depends on the application: for many use cases, the performance of μ P transfer may be sufficient given its lower cost. For scenarios where every fraction of a point in performance is critical, the substantial computational investment in HPO could be justified.

It should be noted that a limitation of the μ P baseline evaluated in this study is that the search for an optimal hyperparameter configuration for the base scale was performed using a grid search over a limited size of 7 points, which is the current usual practice. A more extensive search over the base scale could be used for a stronger conclusion. Additionally, while our μ P implementation correctly applies the scaling rules for initialization, learning rates, and parameter multipliers, we did not tune the individual parameter multipliers (e.g., α_{output} , α_{attn}), which the original work suggests can offer further performance improvements [Yang et al., 2021].

4.3 To Tune or To Scale? An Analysis of Compute Allocation

The most critical strategic question for practitioners with a fixed compute budget is whether to invest it in tuning a model of a certain size or scaling to a larger model. For the model sizes we tested, our results clearly favor scaling.

The "Scaled up model" baseline used a 3x budget to train a larger, 200.6M parameter model with default hyperparameters. Importantly, these defaults were well-chosen and scale robustly, inspired by hyperparameters reported in the open-sci-ref benchmark [Nezhurina et al., 2025a,b], yielding a final loss of 2.59 ± 0.01 . In contrast, the best HPO run (Priorband) used a much larger 6x budget to find the optimal hyperparameters for the smaller 141.8M model, but its best achieved loss was only 2.71 ± 0.00 . Even when extending Priorband’s budget to 20x, the loss did not improve significantly.

This demonstrates that, the performance gain from increasing model scale (from 141.8M to 200.6M parameters) outweighs the gain from extensive HPO on the smaller model. This empirically supports the core principle of scaling laws: model scale is a dominant factor in performance. For practitioners, the recommendation is to prioritize allocating compute resources to increasing model size before investing heavily in HPO.

However, this recommendation depends on where we are on the loss–compute scaling curve [Hoffmann et al., 2022]. As shown in Figure 2, the models in our study operate in the early regime, where increasing model scale yields substantial loss reductions per unit of compute. In the large-scale regime, the loss decreases more slowly with additional compute, meaning that tuning a smaller model could become relatively more cost-effective.

Furthermore, there is a tradeoff between inference cost and performance. Despite HPO consuming more training resources than scaling up the model for a comparable performance, the HPO-trained model is smaller in size; hence, it has a lower inference cost. This may be preferable in the case where inference is performed at large scale.

5 Limitations

This study, while systematic, has several limitations. First, our experiments are conducted on models up to approximately 200 million parameters. While these are nontrivial, it is possible that the dynamics of HPO and the relative benefits of tuning versus scaling could change for models in the 100B+ parameter class (see Section 4.3). Second, our findings are specific to the standard decoder-only Transformer architecture and the SlimPajama dataset [?]. Importantly, the default hyperparameter configuration we used for this architecture is already strong and scale-robust, which may have reduced the apparent gains from HPO. For architectures that have not been studied

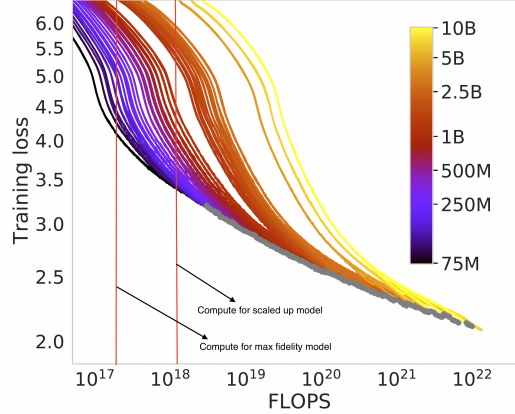


Figure 2: Training loss versus compute across model scales (70M–10B parameters). The envelope represents the minimal loss achievable per FLOP. The red vertical lines indicate the compute used for the scaled-up model and the max-fidelity smaller model in our HPO experiments [Hoffmann et al., 2022].

extensively—such as tabular foundation models [Hollmann et al., 2022], vision-language models, or other specialized architectures—these results may not transfer, and HPO could yield larger improvements. The quantitative results and optimal hyperparameters may also differ for other architectures, such as Mixture-of-Experts (MoE) [Fedus et al., 2022], or for datasets with different quality profiles, an interaction highlighted by Bi et al. [2024]. Finally, our hyperparameter search space, while based on established practices, is not exhaustive. It is conceivable that optimizing other, untested hyperparameters could alter the conclusions.

6 Conclusion

In this work, we presented a principled framework for applying hyperparameter optimization to large language models by using the computational budget as a direct measure of fidelity. Through a rigorous empirical study, we have found that modern multi-fidelity HPO methods, especially prior-informed algorithms like Priorband, can be effective and efficient for pre-training LLMs. These methods offer a measurable, albeit costly, performance advantage over the baseline of μ P hyperparameter transfer with grid search on the smaller base scale. Most importantly, our results offer a strategic recommendation for practitioners: for models within the size range of our study, allocating computational budget towards scaling the model is a more effective strategy for improving performance than allocating it to HPO, provided the model’s size is not fixed.

7 Future Work

This study opens several avenues for future research. Replicating these experiments on models at the scale of tens or hundreds of billions of parameters is a critical next step to verify if the "scaling trumps tuning" conclusion holds in that regime. Our framework could be extended to perform a joint Neural Architecture Search (NAS) and HPO, using compute as a unified budget for exploring both architectural and hyperparameter choices. Investigating the interplay between data quality and optimal hyperparameters could lead to more nuanced scaling strategies. Finally, developing hybrid HPO methods that use μ P transfer to initialize a strong configuration and then use a method like Priorband for focused local search could potentially combine the cost-effectiveness of transfer with the power of search.

Acknowledgements

We would like to express our sincere gratitude to our supervisor, Danny Stoll, for his invaluable guidance, insightful discussions, and continuous support throughout this project. His direction in planning, connecting us with collaborators, and providing crucial learning resources was instrumental

to our work. We also thank Professor Frank Hutter head of the Machine Learning (ML) lab at the University of Freiburg, for fostering a productive research environment and for providing the necessary high-performance computational resources for our experiments.

We are grateful to our colleagues Samir Garibov, Neeratyoy Mallik, David Salinas, and Tarek Abou Chakra for their fruitful discussions and constructive feedback. A special thanks goes to Samir for providing the initial research pipeline, his crucial technical assistance, and the valuable background provided by his thesis.

Finally, we thank the developers of the open-source frameworks we relied on, particularly Nvidia Megatron-LM and NePS, which were essential for our experimental setup.

References

- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, DDL Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 10, 2022.
- Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. *Automated machine learning: methods, systems, challenges*. Springer, 2019.
- Tong Yu and Hong Zhu. Hyper-parameter optimization: A review of algorithms and applications. *arXiv preprint arXiv:2003.05689*, 2020.
- Ge Yang, Edward Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tuning large neural networks via zero-shot hyperparameter transfer. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 17084–17097. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/8df7c2e3c3c3be098ef7b382bd2c37ba-Paper.pdf.
- Marianna Nezhurina, Joerg Franke, Taishi Nakamura, Timur Carstensen, Niccolò Ajroldi, Ville Komulainen, David Salinas, and Jenia Jitsev. Open-sci-ref-0.01 repository. <https://github.com/LAION-AI/open-sci-ref-0.01>, 2025a.
- Marianna Nezhurina, Joerg Franke, Taishi Nakamura, Timur Carstensen, Niccolò Ajroldi, Ville Komulainen, David Salinas, and Jenia Jitsev. Open-sci-ref-0.01: open and reproducible reference baselines for language model and dataset comparison. <https://laion.ai/blog/open-sci-ref-001>, 2025b.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- Danny Stoll, Neeratyoy Mallik, Simon Schrod, Eddie Bergmann, Maciej Janowski, Samir Garibov, Tarek Abou Chakra, Daniel Rogalla, Eddie Bergman, Carl Hvarfner, Ru Binxin, and Frank Hutter. Neural Pipeline Search (NePS), October 2024. URL <https://github.com/automl/neps>.
- Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiusi Du, Zhe Fu, et al. Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954*, 2024.
- J. Bergstra and Y. Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.
- Jasper Snoek, Hugo Larochelle, and Ryan P Adams. Practical bayesian optimization of machine learning algorithms. *Advances in neural information processing systems*, 25, 2012.
- Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of machine learning research*, 18(185):1–52, 2018.

- Neeratyoy Mallik, Edward Bergman, Carl Hvarfner, Danny Stoll, Maciej Janowski, Marius Lindauer, Luigi Nardi, and Frank Hutter. Priorband: Practical hyperparameter optimization in the age of deep learning. *Advances in Neural Information Processing Systems*, 36:7377–7391, 2023.
- Noah Hollmann, Samuel Müller, Katharina Eggersperger, and Frank Hutter. TabPFN: A transformer that solves small tabular classification problems in a second. *arXiv preprint arXiv:2207.01848*, 2022.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

A HPO Search Space Configurations

The hyperparameter search spaces used for the different HPO algorithms are detailed in Tables 2 through 3.

Table 2: Search Space Summary for Priorband, Hyperband, and Random Search. The default values do not affect Random Search.

Hyperparameter	Value Range	Value Space	Default Value
base_max_lr	[0.001, 1.0]	logarithmic	0.004
warmup_fraction	[0.005, 0.05]	uniform	0.05
cooldown_fraction	[0.1, 0.2]	uniform	0.2
adam_beta1	[0.8, 0.99]	uniform	0.9
adam_beta2	[0.9, 0.999]	uniform	0.95
qk_norm		categorical	True
init_std	0.02	constant	–

Table 3: Search Space Summary for Grid Search. The search was conducted over a grid of 7 points.

Hyperparameter	Value Range	Value Space
base_max_lr	[0.001, 1.0]	logarithmic

B Model Architecture Specifications

Our compute-to-architecture framework maps a given compute budget to a specific set of model parameters. Table 4 details the architectures used for the different fidelity levels in the multi-fidelity HPO methods (Priorband, Hyperband) and for the larger model used in the "Scaling Up" baseline. All models use a context size of 1024 and a padded vocabulary size of 50688. Despite variations in the aspect ratio between embedding dimension and layer count, all models adhere to the well-performing range proposed by Kaplan et al. [2020].

Table 4: Architecture summary for models used at different compute fidelities.

Model Size	Compute (FLOPs)	Embedding Dim.	Layers	Heads
<i>Multi-Fidelity HPO Models</i>				
40.3 M	5.1×10^{16}	260	13	2
76.2 M	2.0×10^{17}	420	12	5
141.8 M	8.1×10^{17}	636	12	6
<i>Scaled-Up Single Config Model</i>				
200.6 M	2.4×10^{18}	528	33	4

C Average Runtime for Different Model Sizes

Table 5 shows the average GPU hours required to complete an evaluation for each of the model sizes used across the experiments. All runtimes were measured on NVIDIA L40S GPUs.

Table 5: Average GPU hours per model size.

Model Size	Compute (FLOPs)	Average GPU Hours
40.3M	5.06×10^{16}	2.03
76.2M	2.03×10^{17}	6.38
141.8M	8.10×10^{17}	12.39
200.6M	2.43×10^{18}	27.55

D Extended HPO Trajectories

To investigate the long-term behavior of Priorband, we extended the HPO run to a total budget of 20x the cost of a single max-fidelity evaluation. The resulting incumbent trace is shown in Figure 3. The plot shows that Priorband continues to make marginal progress, finding a slightly better configurations at around 18 and 20 equivalent evaluations. This reinforces its strength as an algorithm that effectively continues to exploit the search space even with a large budget, although the gains diminish over time.

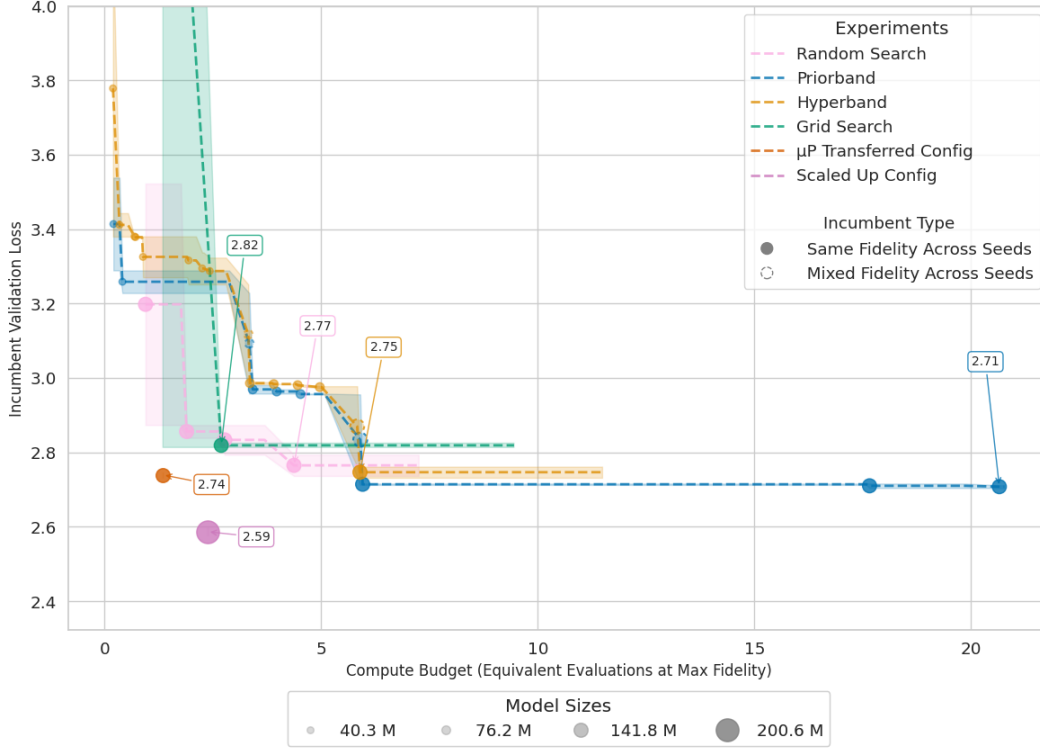


Figure 3: Extended incumbent validation loss trace for Priorband compared to other methods. Priorband finds marginally better configurations with an extended budget.